

Name:

Einführung

Literatur

Tobias Hauser, Florence Maurice, Christian Wenz:

Das Website Handbuch: Programmierung und Design, Markt + Technik Verlag, München 2009

Stefan Münz: Selfhtml 8.1.2, <http://de.selfhtml.org/>

Stefan Münz: HTML5 Handbuch, <http://webkompetenz.wikidot.com/docs:html-handbuch>,
ISBN-13: 978-3645600798

Wikipedia: <http://de.wikipedia.org>

w3schools: <http://www.w3schools.com>

Fragen

Frage 1: Was ist HTML?

HTML ist eine **Auszeichnungssprache** um **Webseiten** zu beschreiben.

HTML bedeutet **Hypertext Markup Language**.

Hypertext bedeutet, dass Texte nicht rein **linear** angeordnet sind, sondern über **Hyperlinks** verbunden sind.

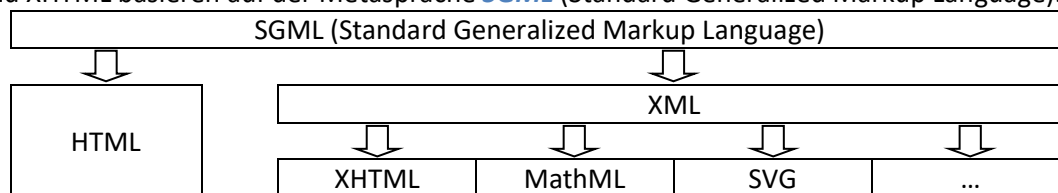
Auszeichnungssprachen dienen dazu, Textinhalte zu **strukturieren (Syntax)** und mit **Bedeutung (Semantik)** zu versehen.

Frage 2: Was bedeutet XHTML?

XHTML bedeutet **eXtensible HyperText Markup Language** (erweiterbares HTML).

Frage 3: Von welchen Sprachen sind HTML und XHTML abgeleitet?

HTML und XHTML basieren auf der Metasprache **SGML** (Standard Generalized Markup Language).



Frage 4: Gibt es einen Unterschied zwischen HTML und HTML5?

HTML5 ist die aktuellste Fassung von HTML. Es gilt als Nachfolger von HTML4 und ersetzt die Standards HTML4.01, XHTML 1.0 und das Document Object Model (DOM) Level 2. Im Gegensatz zu früheren Versionen beinhaltet HTML5 bereits Funktionen für Video, Audio, lokalen Speicher und 2D und 3D Graphiken.

Frage 5 (optional): Was machen die Institutionen W3C und WHATWG? Wer verbirgt sich dahinter?

<http://webkompetenz.wikidot.com/html-handbuch:w3-konsortium>

Frage 6 (optional): Was sind Recommendations? Welche Phasen durchlaufen Sie im Review-Prozess?

<http://webkompetenz.wikidot.com/html-handbuch:w3-konsortium>

Frage 7: Wie ist HTML-aufgebaut? Was ist ein HTML-Element? Was ist ein Tag? Erläutern Sie dies an einem Beispiel.

Ein HTML-Element besteht aus einem **Start**- und **End**-Tag, zwischen denen sich ein **Inhalt** befindet.

Tags werden in **spitze** Klammern gesetzt.

Beispiel:

```
<strong>Mein toller Text</strong>
```

Frage 8: Was ist ein HTML-Standalone-Element? Erläutern Sie dies an einem Beispiel.

Standalone-Tags sind **leere** Elemente und haben **keinen Inhalt**.

Im Beispiel der manuelle (weiche) Zeilenumbruch in HTML (bevorzugt) und XHTML-Notation:

```
Eine Zeile, nachdem ein manueller Zeilenumbruch<br>  
folgt und die nächste Zeile wo noch ein manueller Zeilenumbruch<br />  
folgt.
```

Frage 9: Wie werden Attribute in Tags verwendet? Welchem Zweck dienen sie?

Mit Attributen können in Tags **zusätzliche Angaben** gemacht werden.

Hier signalisiert das Attribut `lang="de"`, dass der Inhalt des Absatzes deutschsprachig ist. Außerdem erscheint mit ein Tool-Tip.

```
<p lang="de" title="toll">Das ist mein <strong>toller</strong> Text</p>
```

Name:

Frage 10: Nennen Sie 3 Eigenschaften, die ein guter Code-Editor haben sollte.

<http://webkompetenz.wikidot.com/html-handbuch:code-editoren>

Syntax-Highlighting: Schlüsselwörter, Klammern, Sonderzeichen, Element- oder Funktionsnamen in Sprachen wie HTML und XML, CSS, JavaScript, PHP, Java, Perl, Python, C, SQL sollte ein Editor farblich hervorheben. Dadurch werden Inhalte wesentlich leichter erfassbar.

Zeilenkontrolle: es sollte je nach Bedarf umschaltbar sein, ob lange Zeilen für die Ansicht umgebrochen werden oder nicht. Ebenso sollten Zeilennummern angezeigt werden können. Denn viele Fehlermeldungen von Validatoren oder Programmiersprachen-Interpretern beziehen sich auf Zeilennummern.

Code-Folding: dieses Feature erlaubt es, strukturierte Codes (z.B. größere Bereiche eines HTML-Dokuments, oder Funktionen eines Scripts) nach Bedarf ein- und auszublenden. Bei umfangreicheren Code-Dateien erleichtert das die Übersicht.

Code-Editier-Hilfen: gute Code-Editoren kennen beispielsweise alle HTML-Element- und -Attributnamen, CSS-Eigenschaften oder PHP-Funktionen. Während des Bearbeitens bieten sie entsprechende Autovervollständigung an, sowie kontext-sensitive Hilfe etwa für die Parameter, die von einer Funktion erwartet werden.

Unterstützende Präzisionsfunktionen: das betrifft etwa eine ausgereifte Suchen-Ersetzen-Funktion, auch über viele Dateien gleichzeitig, und beispielsweise mit der Möglichkeit, reguläre Ausdrücke zu verwenden. Auch sprachentypische Konvertierwerkzeuge wie das Umwandeln von höherwertigen Zeichen in ihre HTML-Entity-Namen oder in numerische HTML-Entities gehören dazu.

Flexible Erweiterbarkeit: viele Code-Editoren speichern ihre Informationen für Syntax-Highlighting und Code-Editier-Hilfen in eigenen Dateien. Diese Dateien haben ein dokumentiertes Format. Dadurch ist es möglich, Syntax-Highlighting-Anweisungen auch für Sprachen zu erstellen, die der Editor bislang nicht unterstützt.

Integration in andere Software-Umgebungen: ein guter Code-Editor ermöglicht es, individuelle Hilfedokumente zu integrieren. Für Entwickler ist es sinnvoll, wenn der Editor direkt mit einem Versionskontrollsystem wie Subversion zusammenarbeitet, oder wenn er es ermöglicht, Dateien direkt via FTP/SFTP von einem Webserver-Rechner zu laden und wieder dort zu speichern.

Zeichenkodierung: Gute Code-Editoren unterstützen alle verbreiteten Formen von Zeichenkodierung, also z.B. ANSI, ISO-8859-x, UTF-8, UTF-16, Unicode-2-Byte, Unicode-4-Byte usw. Dazu gehört auch die Fähigkeit, Inhalte von einer Zeichenkodierung in eine andere zu konvertieren.

Performance: Code-Editoren sind Programme, von denen erwartet wird, dass sie die Ressourcen eines Rechners nicht nennenswert belasten, dass sie sehr schnell laden und sehr robust laufen. Sie sind selbstverständliche Arbeitswerkzeuge, die fast ständig geöffnet sind, auch während anderer Arbeiten.

Übungen und Aufgaben

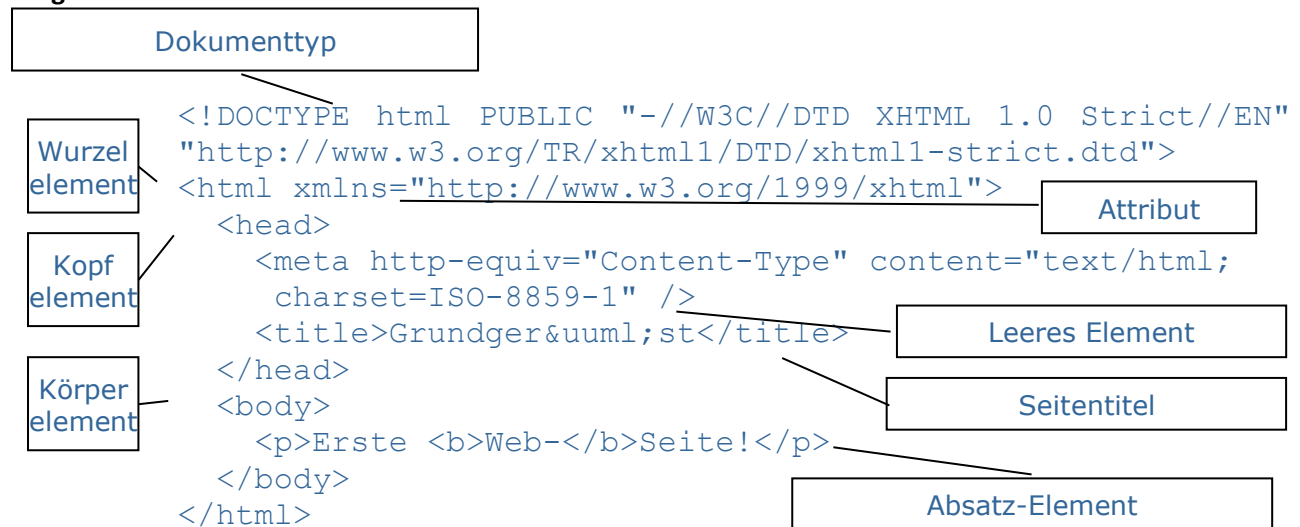
Übung 1: Installation Notepad++

Übung 2: Erste Beispiele programmieren

Dateien:

- 2.2.2a_grundgeruest_html5.html
- 2.2.2b_grundgeruest_html_doctype.html
- 2.2.2c_grundgeruest_xhtml.html
- 2.2_A1_Fehler.html

Aufgabe 1: Kennzeichnen Sie die Elemente im HTML-Dokument.



Aufgabe 2: Welche Fehler werden im HTML-Dokument Fehler.html gemacht? Bessern Sie aus.

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="ISO-8859-1"></meta>
  <title>Ein schöner Tag</title>
</head>
<body>
  <!-- Überschrift -->
  <h1>Ein schöner Tag</h1>
  <!-- Text -->
  <p>Ein schöner Tag war der Tag, an dem ich lernte, was
  <br>ein <strong>Tag<strong> in HTML ist.</p>

  <!-- Fußbereich -->
  <footer><adress>Erna Error, ee@example.org</address></footer>
</body>
</html>
  
```

Aufgabe 3: Arbeiten Sie folgende Kapitel durch.

w3schools

- | | |
|---------------------|----------------|
| • HTML HOME | • HTML Editors |
| • HTML Introduction | • HTML Basic |